

**Министерство науки и высшего образования
Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ» (МОСКОВСКИЙ ПОЛИТЕХ)**

КУРСОВОЙ ПРОЕКТ

По курсу «Основы программирования»

ТЕМА

«Приложение для учёта выданных книг в библиотеке (веб-приложение)»

Выполнил
студент 1 курса
группы 253-325
Волчкова А.М.

Проверил
Красникова И.Н.

МОСКВА 2026

Оглавление

1. ВВЕДЕНИЕ.....	3
1.2. ЦЕЛЬ РАБОТЫ	3
1.3. ЗАДАЧИ РАБОТЫ.....	3
2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	4
2.1. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ	4
2.2. ТРЕБОВАНИЯ К СИСТЕМЕ	5
3. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ.....	7

1. ВВЕДЕНИЕ

Библиотеки играют важную роль в образовании, обеспечивая доступ к книгам и информации, однако с ростом объема литературы и числа читателей учет выданных книг становится все более сложным и требует автоматизации.

Решением является разработка веб-приложения для учета выдачи книг, которое позволяет вести учет в реальном времени, повышая точность и удобство работы для сотрудников и пользователей. Использование веб-технологий обеспечивает доступ к данным из любой точки при наличии интернета, а также упрощает хранение и обработку информации.

Система может содержать информацию о книгах (автор, жанр, год издания), поддерживать поиск по каталогу, регистрацию и авторизацию пользователей, а также уведомления о сроках возврата.

Актуальность работы обусловлена необходимостью повышения эффективности библиотек за счет цифровизации процессов. Цель курсовой работы — разработка информационной системы учета выданных книг. Практическая значимость заключается в автоматизации учета, упрощении работы сотрудников и повышении удобства для читателей.

1.2. ЦЕЛЬ РАБОТЫ

Разработать веб-приложение для учёта выданных книг в библиотеке с использованием Python, Flask и PostgreSQL. Приложение должно позволять полностью контролировать процесс выдачи и возврата книг в библиотеку, в совокупности с контролем в принципе имеющихся книг в библиотеке, контролем сотрудников библиотеки и посетителей.

1.3. ЗАДАЧИ РАБОТЫ

Для достижения поставленной цели необходимо решить следующие задачи:

- исследовать предметную область;
- провести анализ существующих решений;
- спроектировать структуру базы данных;
- разработать структуру системы и её компоненты;
- реализовать приложение;
- провести тестирование и отладку.

Объект исследования: система автоматизации процесса выдачи книг в библиотеке.

Предмет исследования: методы автоматизации учета выданных книг, обработки данных о читателях и организации работы веб-приложения для управления библиотечным фондом.

2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

В условиях цифровизации библиотек веб-приложения для учета книг становятся особенно актуальными. Одним из таких решений является BookTracker – приложение для управления выдачей и возвратом книг с возможностью просмотра статуса и уведомлений для пользователей.

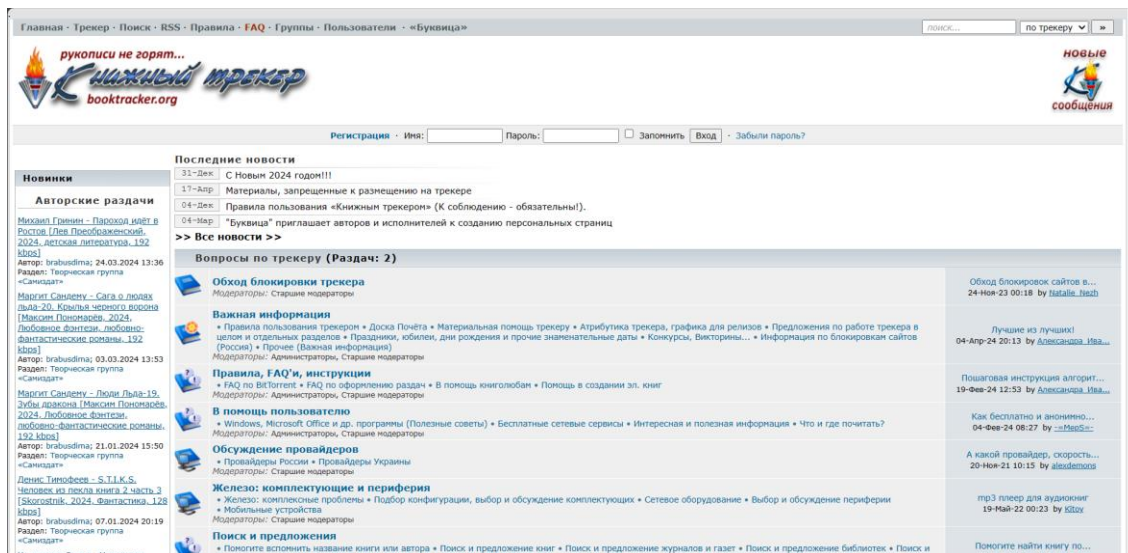


Рисунок 3 – Главная страница приложения BookTracker

Librarian's Assistant – приложение для учета выданных книг, контроля сроков возврата, формирования отчетов и уведомлений о просрочке.

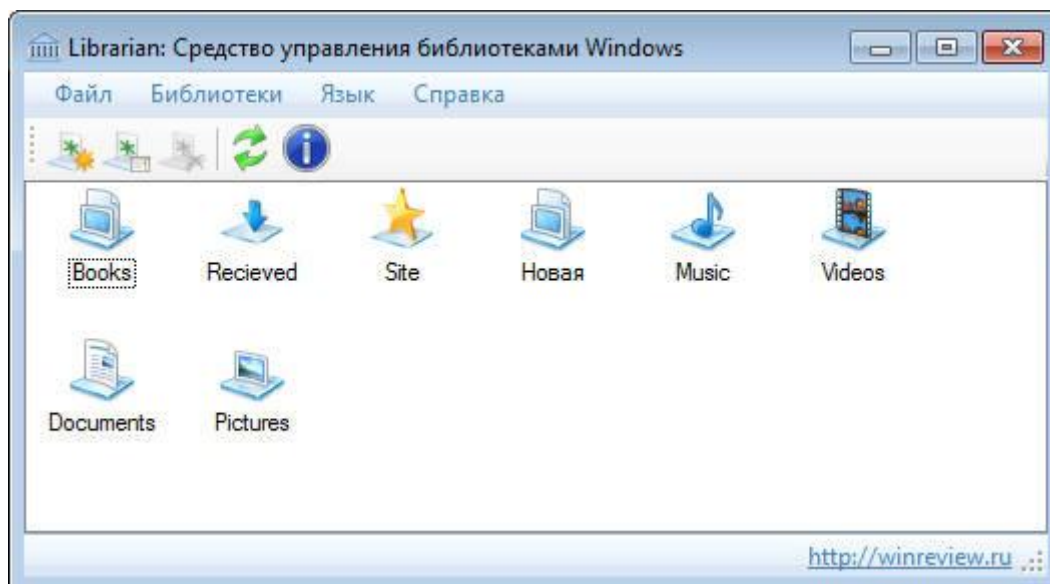


Рисунок 4 – Главная страница приложения Librarian's Assistant

Readers' Shelf – приложение для читателей, позволяющее отслеживать взятые книги, сроки возврата и получать персональные рекомендации по литературе.



Рисунок 5 – Главная страница приложения Readers' Shelf

Существующие веб-приложения для учета книг имеют разные возможности и ограничения. BookTracker предоставляет базовый учет и просмотр данных о книгах, но имеет слабый интерфейс и ограниченную актуальность информации. Librarian's Assistant включает контроль возвратов и уведомления, однако уступает в дизайне и части функций. Readers' Shelf ориентирован на читателей, позволяя просматривать книги и их статус, но почти не подходит для библиотечного учета и не поддерживает административные возможности.

В целом, системы имеют общие недостатки: неполный учет выдачи, слабое управление пользователями и ограниченный функционал уведомлений, что снижает их эффективность.

Анализ показывает необходимость создания более универсального решения, объединяющего основные функции учета, контроля и управления библиотекой.

2.2. ТРЕБОВАНИЯ К СИСТЕМЕ

В процессе разработки и проектирования системы для учёта выданных книг в библиотеке удалось сформулировать требования к разрабатываемому модулю. Приложение должно:

- иметь разный функционал для всех участников процесса работы с системой: админа, библиотекаря и клиента.

Для клиента:

- иметь возможность получать уведомление, когда срок пользования книгой близится к концу/когда просрочена книга;

Для библиотекаря:

- иметь возможность авторизации для библиотекарей, которые записаны в базе данных;
- иметь возможность просматривать список книг и информации о них;
- иметь возможность «выдать книгу», занеся тем самым запись о выдаче в базу данных;
- иметь возможность просматривать список выданных книг вместе со сроками окончания их пользования;
- иметь возможность просматривать сайт;
- иметь возможность возвращать выданную книгу в библиотеку после возврата её клиентом.
- иметь возможность просматривать информацию об авторах и их книгах;
- иметь возможность поиска с помощью фильтрации данных;
- Иметь возможность добавлять новых читателей в библиотеку;
- Иметь возможность редактировать поле читателя «Количество потерянных книг»;

Для администратора:

- иметь возможность пользоваться всем тем же функционалом, что и библиотекарь;
- иметь возможность регистрировать новых работников библиотеки;
- иметь возможность удалять записи о работниках библиотеки, которые по стечению обстоятельств были уволены;
- иметь возможность редактировать записи работников в базе данных;
- иметь возможность добавлять записи о новых книгах в базу данных;
- иметь возможность удалять записи о книгах из базы данных;
- иметь возможность изменять записи о книгах в базе данных;
- иметь возможность авторизоваться по уникальному идентификатору.

3. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

3.1. ER-ДИАГРАММА (КОНЦЕПТУАЛЬНАЯ МОДЕЛЬ)

Для проектирования базы данных в PostgreSQL необходимо определить структуру таблиц и реализовать её с помощью SQL-скриптов. Таблицы взаимодействуют между собой через связи, в частности внешние ключи.

PostgreSQL – мощная СУБД, поддерживающая SQL-стандарты, транзакции, MVCC, подзапросы, триггеры и хранимые процедуры. Для расширенной логики используется PL/pgSQL, позволяющий создавать функции и процедуры.

В результате PostgreSQL является надёжной основой для разработки масштабируемых веб-приложений, хотя может требовать более глубоких знаний SQL по сравнению с использованием фреймворков.

На первом этапе был проведён анализ сущностей базы данных и построена ER-диаграмма, отображающая их связи (Рисунок 6).

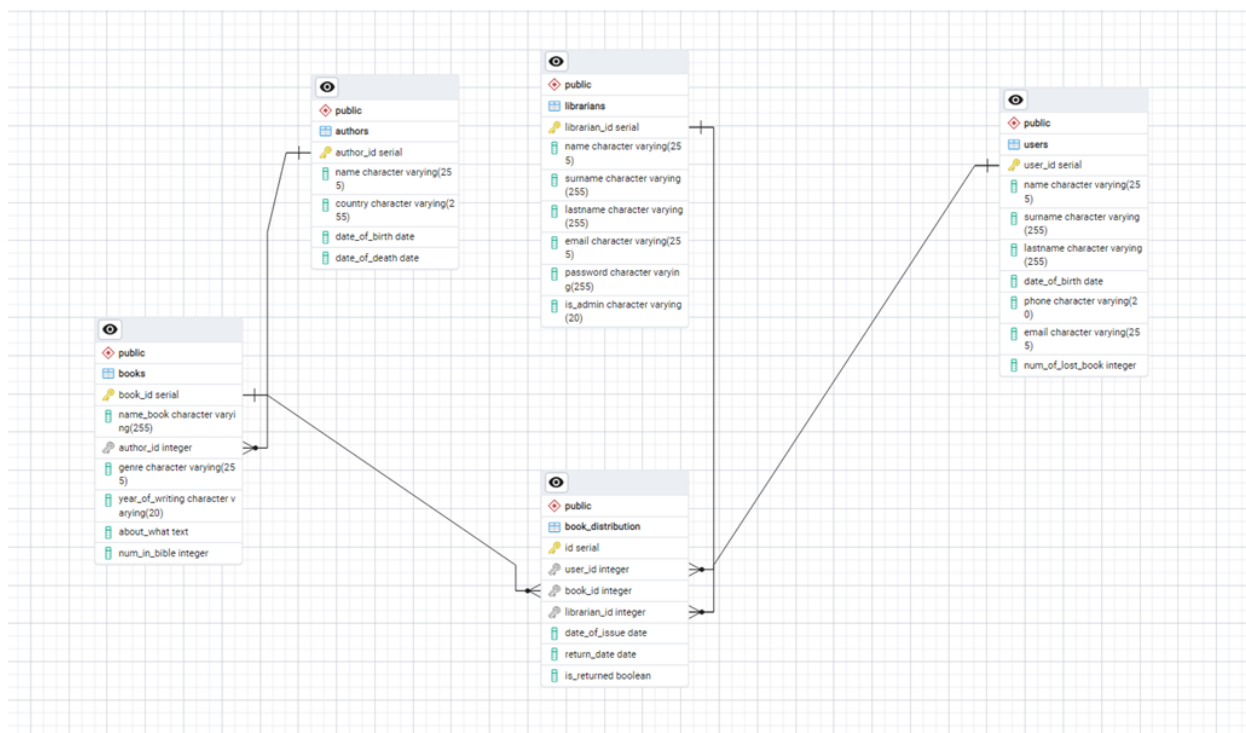


Рисунок 6 – Диаграмма ER-типа

Основываясь на всём вышеперечисленном, была разработана схема баз данных, представленная на рисунке 7.

3.2. ЛОГИЧЕСКАЯ МОДЕЛЬ ДАННЫХ

Опираясь на разработанную схему базы данных, были описаны структуры таблиц, которые будут представлены ниже.

Таблица 1 – Структура таблицы users

Наименование поля	Тип	По умолчанию	Ключ	Описание
user_id	integer		РК	Идентификатор читателя, посещающего библиотеку
name	character varying (255)			Имя читателя
surname	character varying (255)			Фамилия читателя
lastname	character varying (255)			Отчество читателя
date_of_birth	date			Дата рождения читателя
phone	character varying (20)			Телефон читателя
email	character varying (255)			Почта читателя
num_of_lost_book	integer			Количество потерянных книг читателем

Таблица 2 – Структура таблицы authors

Наименование поля	Тип	По умолчанию	Ключ	Описание
author_id	integer		РК	Идентификатор писателя
name	character varying (255)			ФИО писателя
country	character varying (255)			Страна рождения писателя
date_of_birth	date			Дата рождения писателя
date_of_death	date	null		Дата смерти писателя

Таблица 3 – Структура таблицы books

Наименование поля	Тип	По умолчанию	Ключ	Описание
book_id	integer		РК	Идентификатор книги
name_book	character varying (255)			Название книги
author_id	integer		FK	Идентификатор автора, написавшего книгу
genre	character varying (255)			Жанр книги
year_of_writing	character varying (20)			Год написания книги

about_what	text			Короткое описание книги
num_in_bible	integer			Количество данного произведения в библиотеке

Таблица 4 – Структура таблицы librarians

Наименование поля	Тип	По умолчанию	Ключ	Описание
librarian_id	integer		PK	Идентификатор библиотекаря
name	character varying (255)			Имя библиотекаря
surname	character varying (255)			Фамилия библиотекаря
lastname	character varying (255)			Отчество библиотекаря
email	character varying (255)			Почта библиотекаря
password	character varying (255)			Пароль для входа библиотекаря
is_admin	character varying (20)			роль сотрудника в системе

Таблица 5 – Структура таблицы book_distribution

Наименование поля	Тип	По умолчанию	Ключ	Описание
id	integer		PK	Идентификатор записи
user_id	integer		FK	Идентификатор читателя, взявшего книгу
book_id	integer		FK	Идентификатор книги, которую взяли
librarian_id	integer		FK	Идентификатор библиотекаря, который выдал книгу
date_of_issue	date			Дата выдачи книги
return_date	date			Дата возврата
is_returned	boolean			Отметка, была ли возвращена книга в указанный срок возврата

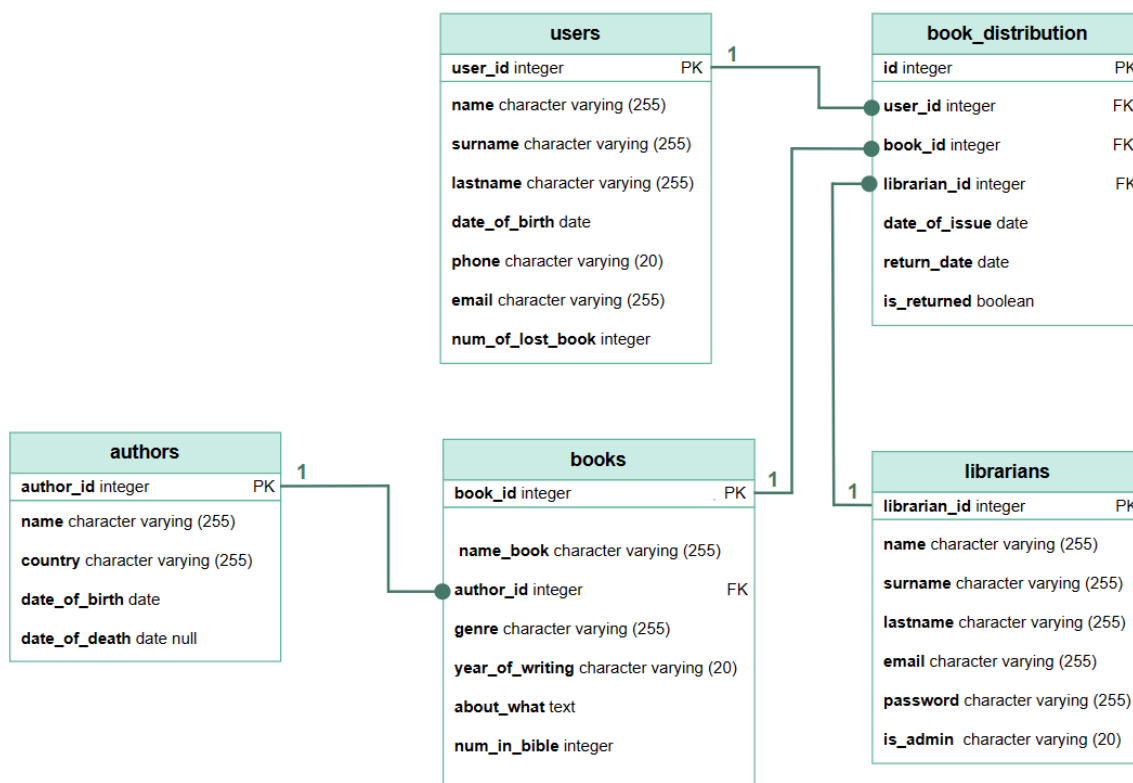


Рисунок 7 – Схема базы данных

Качество информационной системы во многом определяется базой данных. Важно правильно выбрать типы данных, настроить связи между таблицами и выполнить нормализацию для обеспечения быстрого доступа и безопасности информации.

Стабильная структура БД повышает устойчивость системы и упрощает её развитие, обеспечивая эффективную обработку данных.

4. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

4.1. АРХИТЕКТУРА И СТЕК ТЕХНОЛОГИЙ

Приложение реализовано на языке Python 3.8+ с использованием только стандартных библиотек:

Таблица 6 – Таблица стека технологий

Компонент	Назначение
Flask / Jinja2	Веб-фреймворк и шаблонизатор
sqlite3	Локальная реляционная база данных
datetime	Работа с датами и периодами
os / json	Вспомогательные системные операции

Архитектура приложения реализована по паттерну MVC (Model-View-Controller): работа с базой данных выполняется в Model, маршрутизация – в Controller (Flask), а отображение интерфейса – во View с использованием шаблонов Jinja2. Основная логика сосредоточена в файле server.py, а шаблоны интерфейса размещены в папке frontend/, что упрощает сопровождение и расширение системы.

4.2. СТРУКТУРА МОДУЛЕЙ И ЛИСТИНГ ПРОГРАММЫ

В соответствии с указанными ранее требованиями, были разработаны модули информационной системы, структура которых отображена на иллюстрации 8.



Рисунок 8 – Структура информационной системы

Информационная система, разработка которой является основной задачей проекта, будет состоять из 8 взаимосвязанных блоков, также именуемых модулями, каждый из которых выполняет свою определенную функцию.

Листинг программы представлен в приложении 1.

4.3. КЛЮЧЕВЫЕ АЛГОРИТМЫ

Алгоритм контроля выданных книг: при открытии раздела учета система выполняет запрос к базе данных, получает информацию о выданных книгах, сроках возврата и статусе выдачи. Далее данные проверяются по заданным условиям: наличие просроченных книг, приближение даты возврата и факт возврата книги. На основе этого формируются уведомления и обновляется статус книги в системе.

Алгоритм навигации: каждый раздел веб-приложения реализован как отдельный URL-маршрут Flask. При переходе пользователя браузер отправляет HTTP GET-запрос, после чего сервер обрабатывает данные из базы и отображает соответствующий HTML-шаблон Jinja2 с актуальной информацией.

5. SQL-ЗАПРОСЫ К БАЗЕ ДАННЫХ

В приложении реализованы следующие ключевые запросы:

```
CREATE TABLE Users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  surname VARCHAR(255) NOT NULL,
  lastname VARCHAR(255) NOT NULL,
  date_of_birth DATE NOT NULL,
  phone VARCHAR(20) NOT NULL,
  email VARCHAR(255) NOT NULL,
  num_of_lost_book INT DEFAULT 0 NOT NULL
);

CREATE TABLE Authors (
  author_id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  country VARCHAR(255) NOT NULL,
  date_of_birth DATE NOT NULL,
  date_of_death DATE
);

CREATE TABLE Books (
  book_id SERIAL PRIMARY KEY,
  Name_book VARCHAR(255) NOT NULL,
  Author_id INT REFERENCES Authors(author_id) ON UPDATE CASCADE ON DELETE CASCADE NOT NULL,
  Genre VARCHAR(255) NOT NULL,
  Year_of_writing VARCHAR(20) NOT NULL,
  About_what TEXT NOT NULL,
  Num_in_bible INT NOT NULL
);

CREATE TABLE Librarians (
  librarian_id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  surname VARCHAR(255) NOT NULL,
  lastname VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL,
  password VARCHAR(255) NOT NULL,
  is_admin VARCHAR(20) DEFAULT 'user' NOT NULL
);

CREATE TABLE Book_distribution (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES Users(user_id) ON UPDATE CASCADE ON DELETE CASCADE NOT NULL,
  book_id INT REFERENCES Books(book_id) ON UPDATE CASCADE ON DELETE CASCADE NOT NULL,
  librarian_id INT REFERENCES Librarians(librarian_id) ON UPDATE CASCADE ON DELETE CASCADE NOT NULL,
  date_of_issue DATE default current_date,
  return_date DATE NOT NULL,
  is_returned BOOLEAN DEFAULT FALSE NOT NULL
);
```

Рис. 9 – запрос создания базы данных

```
'SELECT * FROM librarians WHERE email = $1'
```

Рис. 10 – запрос поиска пользователей по email

```
'select librarians.email from librarians where email = $1'
```

Рис. 11 – запрос поиска библиотекарей

```
`insert into librarians (name, surname, lastname, email, password) values ($1, $2, $3, $4, $5) Returning *`
```

Рис. 12 – запрос добавления библиотекарей

```
'DELETE FROM librarians WHERE librarian_id=$1'
```

Рис. 12 – запрос удаления библиотекарей

```
`UPDATE users SET num_of_lost_book = num_of_lost_book + 1 WHERE user_id=
```

Рис. 12 – запрос записи потерянных книг

```
`UPDATE book_distribution SET is_returned=true WHERE id=$
```

Рис. 13 – запрос возврата потерянных книг

```
`insert into users (name, surname, lastname, date_of_birth, phone, email) values ($1, $2, $3, $4, $5, $6) Returning *`
```

Рис. 14 – запрос добавления читателя

```
`select book_distribution.id, book_distribution.user_id, users.surname as users, books.name_book as books, book_distribution.librarian_id, book_distribution.date_of_issue, book_distribution.return_date from book_distribution join users on book_distribution.user_id = users.user_id join books on book_distribution.book_id = books.book_id where book_distribution.is_returned = false`
```

Рис. 15 – запрос вывода списка выданных книг (очень длинный, поэтому представлен ниже)

```
select book_distribution.id, book_distribution.user_id, users.surname as users,  
books.name_book as books, book_distribution.librarian_id,  
book_distribution.date_of_issue, book_distribution.return_date from book_distribution  
join users on book_distribution.user_id = users.user_id join books on  
book_distribution.book_id = books.book_id where book_distribution.is_returned =  
false`
```

И прочие, основная масса запросов реализована по уже указанному типу.

6. ТЕСТИРОВАНИЕ

6.1. ФУНКЦИОНАЛЬНОЕ ТЕСТИРОВАНИЕ

Важным этапом разработки информационной системы является функциональное тестирование, позволяющее проверить корректность, стабильность и соответствие приложения установленным требованиям. Проведение тестирования помогает выявить ошибки и недочёты на ранних этапах разработки, что повышает качество и надёжность конечного продукта.

В ходе работы было проведено функциональное тестирование системы учета выданных книг в библиотеке, включающее:

- проверку авторизации пользователей;
- проверку общего функционала сотрудников библиотеки;
- проверку административного функционала системы.

Таблица 8 – Результаты тестирования функционала авторизации

Название функции	Входные данные	Ожидаемый результат	Полученный результат	Вывод
Авторизация пользователя – сотрудника	Отправка формы с корректными данными: 1. Ввод данных в поле «Email»: skugarova_a23@gmail.com 2. Ввод данных в поле «Пароль»: 11111 3. Нажатие на кнопку «Войти»	Авторизация проходит успешно, пользователь перенаправлен в раздел сотрудника	Авторизация проходит успешно, пользователь перенаправлен в раздел сотрудника	Функция работает верно
Авторизация пользователя – администратора	Отправка формы с корректными данными: 1. Ввод данных в поле «Email»: volarmak@gmail.com 2. Ввод данных в поле «Пароль»: 55555 3. Нажатие на кнопку «Войти»	Авторизация проходит успешно, пользователь перенаправлен в раздел администратора	Авторизация проходит успешно, пользователь перенаправлен в раздел администратора	Функция работает верно
Проверка данных при авторизации пользователя	Отправка формы с несуществующим логином: 1. Ввод данных в поле «Email»: takogoNet@gmail.com 2. Ввод данных в поле «Пароль»: 12345 3. Нажатие на кнопку «Войти»	Появление ошибки о неправильном введении почты	Появление ошибки о неправильном введении почты «Ошибка при входе: Пользователь с таким email не найден»	Функция работает верно
Проверка данных при	Отправка формы с неправильным паролем:	Появление ошибки о	Появление ошибки о неправильном	Функция работает верно

авторизации пользователя	1. Ввод данных в поле «Email»: skugarova_a23@gmail.com 2. Ввод данных в поле «Пароль»: 11166 3. Нажатие на кнопку «Войти»	неправильном введении пароля	введении пароля «Ошибка при входе: Неверный пароль»	
Проверка данных при авторизации пользователя	Отправка формы с email некорректного формата: 1. Ввод данных в поле «Email»: 11166 2. Ввод данных в поле «Пароль»: 11166 3. Нажатие на кнопку «Войти»	Появление ошибки о неверном формате логина	Появление ошибки о неверном формате логина «Адрес электронной почты должен содержать символ «@». Похоже, вы его забыли: 11166»	Функция работает верно
Проверка данных при авторизации пользователя	Отправка незаполненной формы: 1. Ввод данных в поле «Email»: 2. Ввод данных в поле «Пароль»: 3. Нажатие на кнопку «Войти»	Появление ошибки о том, что нужно заполнить поля	Появление ошибки о том, что нужно заполнить поля «Вы пропустили это поле»	Функция работает верно
Проверка данных при авторизации пользователя	Отправка формы с незаполненной почтой: 1. Ввод данных в поле «Email»: 2. Ввод данных в поле «Пароль»: 11111 3. Нажатие на кнопку «Войти»	Появление ошибки о том, что нужно заполнить поле почты	Появление ошибки о том, что нужно заполнить поле почты «Вы пропустили это поле»	Функция работает верно
Проверка данных при авторизации пользователя	Отправка формы с незаполненным паролем: 1. Ввод данных в поле «Email»: skugarova_a23@gmail.com 2. Ввод данных в поле «Пароль»: 3. Нажатие на кнопку «Войти»	Появление ошибки о том, что нужно заполнить поле пароля	Появление ошибки о том, что нужно заполнить поле пароля «Вы пропустили это поле»	Функция работает верно
Проверка данных при авторизации пользователя	Отправка формы с почтой отличного от @gmail.com домена: 1. Ввод данных в поле «Email»: a.ivanov@example.com 2. Ввод данных в поле «Пароль»: 22222	Авторизация проходит успешно, пользователь перенаправлен в соответствующий его роли раздел	Авторизация проходит успешно, пользователь перенаправлен в соответствующий его роли раздел	Функция работает верно

	3. Нажатие на кнопку «Войти»			
--	---------------------------------	--	--	--

6.2. НЕФУНКЦИОНАЛЬНОЕ ТЕСТИРОВАНИЕ

Производительность: время обработки запросов системы учета книг составило 40–80 мс (Intel Core i5, 8 ГБ RAM), включая выполнение SQL-запросов и загрузку данных из базы. Полное отображение страницы в браузере занимало 150–200 мс.

Кроссплатформенность: веб-приложение успешно функционирует на Windows 10/11, Ubuntu 22.04 и macOS Ventura.

7. ЗАКЛЮЧЕНИЕ

Результатом курсового проектирования стало разработанное приложение для учета выданных книг в библиотеке, что подтверждает достижение поставленной цели. Система предназначена для библиотек и отдельных пользователей и позволяет вести учет книг, читателей и выдач, а также управлять возвратами, отслеживать потери и отправлять уведомления о просроченных книгах.

Сотрудники библиотеки после авторизации могут просматривать информацию о книгах, авторах и читателях, добавлять новых пользователей, выдавать и принимать книги. Администратор обладает расширенными правами и может изменять все данные в базе.

В ходе разработки был проведен анализ предметной области, на основе которого построены UML-диаграммы (последовательности и вариантов использования), отражающие логику работы системы и роли пользователей. Также были сформированы требования к ПО, выбраны технологии, разработана ER-диаграмма и структура базы данных, описаны основные алгоритмы и реализованы ключевые модули системы.

Дополнительно была подготовлена пользовательская документация и проведено тестирование с формированием и прохождением тестовых сценариев.

Главным преимуществом системы является автоматизация библиотечных процессов для сотрудников и администраторов, что существенно упрощает управление библиотекой.

В дальнейшем планируется доработка функционала, включая завершение незаконченных модулей, улучшение интерфейса по отзывам пользователей, расширение ролей системы, добавление новых возможностей (например, отметка любимых книг, выдача нескольких книг в одной операции, формирование статистики и отчетов), а также улучшение адаптивности и производительности приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Python для начинающих, Дж. Бриггс, Python для детей и начинающих: М.: Эксмо, 2019.
2. В.П. Рядченко, Методическое пособие по выполнению лабораторных работ.
3. Прографируем на Python, М. Доусон, Прографируем на Python: СПб.: Питер, 2021.
4. Python. К вершинам мастерства, Л. Рамальо, Python. К вершинам мастерства: СПб.: Питер, 2021.
5. Изучаем Python, М. Лутц, Изучаем Python, 5-е изд.: СПб.: Питер, 2020.
6. Python Software Foundation. Python 3 Documentation. URL: <https://docs.python.org/3/> (дата обращения: 15.05.2025).
7. SQLite Documentation. URL: <https://www.sqlite.org/docs.html> (дата обращения: 15.05.2025).
8. Flask Documentation. URL: <https://flask.palletsprojects.com/> (дата обращения: 15.05.2025).

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    port: int = 3000
    db_path: str = "bookshelf.db"
    jwt_secret: str = "123321"
    cors_origins: str = "*"

settings = Settings()

from pathlib import Path

import aiosqlite

from .config import settings

_conn: aiosqlite.Connection | None = None
_INIT_SQL = Path(__file__).parent / "init.sql"

async def connect() -> None:
    global _conn
    _conn = await aiosqlite.connect(settings.db_path)
    _conn.row_factory = aiosqlite.Row
    await _conn.execute("PRAGMA foreign_keys = ON")
    cur = await _conn.execute(
        "SELECT name FROM sqlite_master WHERE type='table' AND name='librarians'"
    )
    has_tables = await cur.fetchone()
    await cur.close()
    if not has_tables:
        await _conn.executescript(_INIT_SQL.read_text(encoding="utf-8"))
        await _conn.commit()

async def disconnect() -> None:
    global _conn
    if _conn is not None:
        await _conn.close()
        _conn = None

def conn() -> aiosqlite.Connection:
    if _conn is None:
```

```
        raise RuntimeError("DB not initialized")
    return _conn

from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles

from .config import settings
from .db import conn, connect, disconnect
from .routers import auth, authors, books, issue, readers

@asynccontextmanager
async def lifespan(app: FastAPI):
    await connect()
    await conn().execute("SELECT 1")
    yield
    await disconnect()

app = FastAPI(title="BookShelf", lifespan=lifespan)

origins = ["*"] if settings.cors_origins == "*" else [o.strip() for o in
settings.cors_origins.split(",")]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/health")
async def health() -> dict:
    return {"ok": True}

app.include_router(auth.router)
app.include_router(books.router)
app.include_router(authors.router)
app.include_router(readers.router)
app.include_router(issue.router)

app.mount("/frontend", StaticFiles(directory="public/frontend", html=True),
name="frontend")
app.mount("/backend", StaticFiles(directory="public/backend"), name="backend")
from fastapi import APIRouter
```

```
from fastapi.responses import JSONResponse, PlainTextResponse

from ..db import conn
from ..schemas.auth import LoginIn, RegisterIn
from ..security import generate_token, hash_password, verify_password

router = APIRouter()

@router.post("/login")
async def login(payload: LoginIn):
    try:
        cur = await conn().execute(
            "SELECT * FROM librarians WHERE email = ?", (payload.email,)
        )
        user = await cur.fetchone()
        await cur.close()

        if user is None:
            return PlainTextResponse("Пользователь с таким email не найден",
status_code=404)

        if not verify_password(payload.password, user["password"]):
            return PlainTextResponse("Неверный пароль", status_code=401)

        token = generate_token(payload.email, user["librarian_id"], user["is_admin"])
        return {"token": token, "userId": user["librarian_id"], "role":
user["is_admin"]}
    except Exception as e:
        print(f"Ошибка при входе в аккаунт: {e}")
        return PlainTextResponse("Что-то пошло не так", status_code=500)

@router.post("/register")
async def register(payload: RegisterIn):
    try:
        cur = await conn().execute(
            "SELECT email FROM librarians WHERE email = ?", (payload.email,)
        )
        maybe_user = await cur.fetchone()
        await cur.close()
        if maybe_user is not None:
            return JSONResponse(
                {"message": "Такой сотрудник уже есть уже существует"},
status_code=400
            )

        hashed = hash_password(payload.password)
        cur = await conn().execute(
            "INSERT INTO librarians (name, surname, lastname, email, password) "
```

```
        "VALUES (?, ?, ?, ?, ?) RETURNING *",
        (payload.name, payload.surname, payload.lastname, payload.email, hashed),
    )
    new_libr = await cur.fetchone()
    await cur.close()
    await conn().commit()
    return dict(new_libr) if new_libr is not None else None
except Exception as e:
    print(e)
    return JSONResponse({"message": "Ошибка при добавлении"}, status_code=400)

async def _delete_librarian(librarian_id: int):
    try:
        await conn().execute(
            "DELETE FROM librarians WHERE librarian_id = ?", (librarian_id,)
        )
        await conn().commit()
        return {"message": "Libr deleted successfully"}
    except Exception as e:
        print(e)
        return PlainTextResponse("Error executing query", status_code=500)

@router.delete("/deletel/{librarian_id}")
async def delete_librarian(librarian_id: int):
    return await _delete_librarian(librarian_id)

@router.delete("/deletelibr/{librarian_id}")
async def delete_librarian_alias(librarian_id: int):
    return await _delete_librarian(librarian_id)
```

```
import jwt
from fastapi.testclient import TestClient

from src.config import settings
from src.main import app

def test_register_creates_librarian() -> None:
    with TestClient(app) as client:
        response = client.post(
            "/register",
            json={
                "name": "Test",
                "surname": "User",
                "lastname": "X",
```

```
        "email": "test@example.com",
        "password": "pass123",
    },
)
assert response.status_code == 200
body = response.json()
assert body["email"] == "test@example.com"
assert body["password"] != "pass123"
assert body["password"].startswith("$2")

def test_register_duplicate_email_returns_400() -> None:
    with TestClient(app) as client:
        payload = {
            "name": "A",
            "surname": "B",
            "lastname": "C",
            "email": "dup@example.com",
            "password": "p",
        }
        client.post("/register", json=payload)
        response = client.post("/register", json=payload)
        assert response.status_code == 400
        assert response.json()["message"] == "Такой сотрудник уже есть уже существует"

def test_login_unknown_email_returns_404() -> None:
    with TestClient(app) as client:
        response = client.post(
            "/login", json={"email": "nobody@example.com", "password": "x"}
        )
        assert response.status_code == 404
        assert response.text == "Пользователь с таким email не найден"

def test_login_wrong_password_returns_401() -> None:
    with TestClient(app) as client:
        client.post(
            "/register",
            json={
                "name": "A",
                "surname": "B",
                "lastname": "C",
                "email": "u@example.com",
                "password": "right",
            },
        )
        response = client.post(
            "/login", json={"email": "u@example.com", "password": "wrong"}
```

```
)
assert response.status_code == 401
assert response.text == "Неверный пароль"

def test_login_success_returns_token_and_role() -> None:
    with TestClient(app) as client:
        reg = client.post(
            "/register",
            json={
                "name": "A",
                "surname": "B",
                "lastname": "C",
                "email": "ok@example.com",
                "password": "good",
            },
        )
        librarian_id = reg.json()["librarian_id"]

        response = client.post(
            "/login", json={"email": "ok@example.com", "password": "good"}
        )
        assert response.status_code == 200
        body = response.json()
        assert body["userId"] == librarian_id
        assert body["role"] == "user"

        payload = jwt.decode(body["token"], settings.jwt_secret,
                                algorithms=["HS256"])
        assert payload["email"] == "ok@example.com"
        assert payload["userId"] == librarian_id
        assert payload["role"] == "user"

def test_delete_librarian_both_aliases() -> None:
    with TestClient(app) as client:
        reg1 = client.post(
            "/register",
            json={"name": "A", "surname": "B", "lastname": "C",
                "email": "d1@example.com", "password": "p"},
        )
        reg2 = client.post(
            "/register",
            json={"name": "A", "surname": "B", "lastname": "C",
                "email": "d2@example.com", "password": "p"},
        )
        id1 = reg1.json()["librarian_id"]
        id2 = reg2.json()["librarian_id"]

        r1 = client.delete(f"/deletel/{id1}")
```

```
    assert r1.status_code == 200
    assert r1.json() == {"message": "Libr deleted successfully"}

    r2 = client.delete(f"/deletelibr/{id2}")
    assert r2.status_code == 200
    assert r2.json() == {"message": "Libr deleted successfully"}

from fastapi.testclient import TestClient

from src.main import app

def test_list_authors_returns_seeded_rows() -> None:
    with TestClient(app) as client:
        response = client.get("/authors")
        assert response.status_code == 200
        rows = response.json()
        assert len(rows) == 13
        assert set(rows[0].keys()) == {
            "author_id", "name", "country", "date_of_birth", "date_of_death",
        }

def test_delete_author_cascades_to_books() -> None:
    with TestClient(app) as client:
        before = client.get("/").json()
        assert any(b["author"] == "Льюис Кэрролл" for b in before)

        r = client.delete("/authors/11")
        assert r.status_code == 200

        after_authors = client.get("/authors").json()
        assert all(a["author_id"] != 11 for a in after_authors)

        after_books = client.get("/").json()
        assert all(b["author"] != "Льюис Кэрролл" for b in after_books)

from fastapi.testclient import TestClient

from src.main import app

def test_list_books_returns_seeded_rows() -> None:
    with TestClient(app) as client:
        response = client.get("/")
        assert response.status_code == 200
        rows = response.json()
        assert len(rows) == 11
        first = rows[0]
        assert set(first.keys()) == {
```

```
        "book_id", "name_book", "author", "genre",
        "year_of_writing", "about_what", "num_in_bible",
    }

def test_list_books_join_resolves_author() -> None:
    with TestClient(app) as client:
        rows = client.get("/").json()
        alice = next(r for r in rows if r["name_book"] == "Алиса в стране чудес")
        assert alice["author"] == "Льюис Кэрролл"

def test_one_book_returns_single() -> None:
    with TestClient(app) as client:
        response = client.get("/oneBook/1")
        assert response.status_code == 200
        body = response.json()
        assert body["book_id"] == 1
        assert body["name_book"] == "Алиса в стране чудес"

def test_one_book_missing_returns_null() -> None:
    with TestClient(app) as client:
        response = client.get("/oneBook/9999")
        assert response.status_code == 200
        assert response.json() is None

def test_delete_book_removes_row() -> None:
    with TestClient(app) as client:
        r = client.delete("/delete/11")
        assert r.status_code == 200
        assert r.json() == {"message": "Book deleted successfully"}

        remaining = client.get("/").json()
        assert all(row["book_id"] != 11 for row in remaining)

def test_give_book_without_id_in_path_still_works() -> None:
    with TestClient(app) as client:
        response = client.post(
            "/give",
            json={"book_id": 1, "user_id": 1, "libr_id": 1, "return_date": "2030-12-31"},
        )
        assert response.status_code == 200
        assert isinstance(response.json(), list)
        assert len(response.json()) == 1

def test_give_book_inserts_distribution() -> None:
```

```
with TestClient(app) as client:
    response = client.post(
        "/give/1",
        json={
            "book_id": 1,
            "user_id": 1,
            "libr_id": 1,
            "return_date": "2030-12-31",
        },
    )
    assert response.status_code == 200
    body = response.json()
    assert isinstance(body, list)
    assert len(body) == 1
    assert body[0]["user_id"] == 1
    assert body[0]["book_id"] == 1
    assert body[0]["librarian_id"] == 1
    assert body[0]["return_date"] == "2030-12-31"
    assert body[0]["is_returned"] == 0

from fastapi.testclient import TestClient

from src.main import app

def test_list_issue_returns_unreturned_only() -> None:
    with TestClient(app) as client:
        response = client.get("/issue")
        assert response.status_code == 200
        rows = response.json()
        assert len(rows) == 12
        assert set(rows[0].keys()) == {
            "id", "user_id", "users", "books",
            "librarian_id", "date_of_issue", "return_date",
        }

def test_return_issue_marks_returned() -> None:
    with TestClient(app) as client:
        r = client.put("/issue/1")
        assert r.status_code == 200
        assert r.json() == []

        remaining = client.get("/issue").json()
        assert all(row["id"] != 1 for row in remaining)
        assert len(remaining) == 11

def test_list_librarians_returns_seeded_rows() -> None:
    with TestClient(app) as client:
```

```
response = client.get("/librarians")
assert response.status_code == 200
rows = response.json()
assert len(rows) == 5
assert set(rows[0].keys()) == {
    "librarian_id", "name", "surname", "lastname",
    "email", "password", "is_admin",
}
```